

1. Introduction to Deep Learning

1.1 What is Deep Learning?

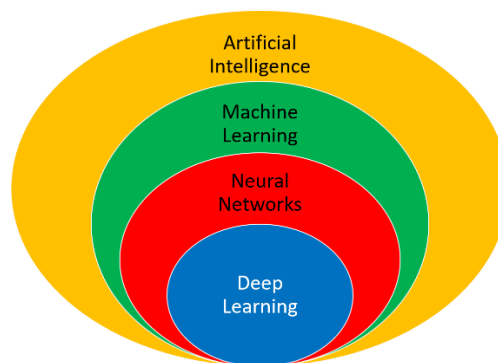
Deep learning is a subset of machine learning that deals with algorithms inspired by the structure and function of the human brain, known as artificial neural networks. It aims to automatically learn representations of data through multiple layers of abstraction.

1.1.1 Definition and overview

Deep learning can be defined as a class of machine learning techniques that use artificial neural networks with multiple layers of representation. It has shown remarkable success in various fields such as computer vision, natural language processing, and speech recognition.

1.1.2 Differences between deep learning and traditional machine learning

- **Representation of Data:** Deep learning automatically learns hierarchical representations of data, whereas traditional machine learning often requires manual feature extraction.
- **Performance:** Deep learning models tend to perform exceptionally well on tasks with large amounts of data, surpassing traditional methods in many domains.
- **Computational Requirements:** Deep learning typically requires more computational resources due to its complex architecture and training processes.



1.1.3 Applications of deep learning

Deep learning finds applications in diverse fields:

- **Image Recognition:** Classifying and detecting objects in images.
- **Natural Language Processing (NLP):** Processing and understanding human language.
- **Speech Recognition:** Converting spoken language into text or commands.
- **Medical Diagnosis:** Analyzing medical images and data for diagnostics.
- **Autonomous Vehicles:** Enabling self-driving cars to perceive and react to the environment.

Machine Learning	Deep Learning
Apply statistical algorithms to learn the hidden patterns and relationships in the dataset.	Uses artificial neural network architecture to learn the hidden patterns and relationships in the dataset.
Can work on the smaller amount of dataset	Requires the larger volume of dataset compared to machine learning
Better for the low-label task.	Better for complex task like image processing, natural language processing, etc.
Takes less time to train the model.	Takes more time to train the model.
A model is created by relevant features which are manually extracted from images to detect an object in the image.	Relevant features are automatically extracted from images. It is an end-to-end learning process.
Less complex and easy to interpret the result.	More complex, it works like the black box interpretations of the result are not easy.
It can work on the CPU or requires less computing power as compared to deep learning.	It requires a high-performance computer with GPU.

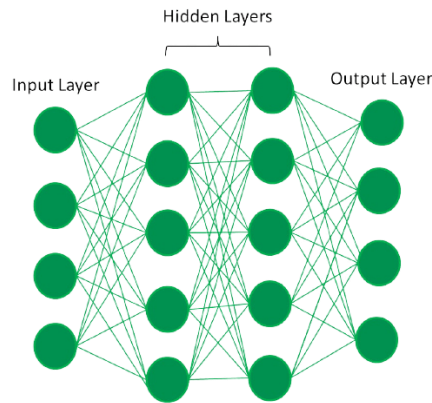
1.2 Key Concepts in Deep Learning

1.2.1 Neural networks

Neural networks are computational models inspired by the biological neural networks of the human brain. They consist of interconnected nodes (neurons) organized in layers.

1.2.2 Layers (input, hidden, output)

- **Input Layer:** Receives input data and passes it to the next layer.
- **Hidden Layers:** Intermediate layers between input and output, where computation occurs.
- **Output Layer:** Produces the final output or prediction.



1.2.3 Activation functions

Activation functions introduce non-linearity into the output of a neuron, allowing neural networks to model complex relationships in data.

1.2.4 Forward and backward propagation

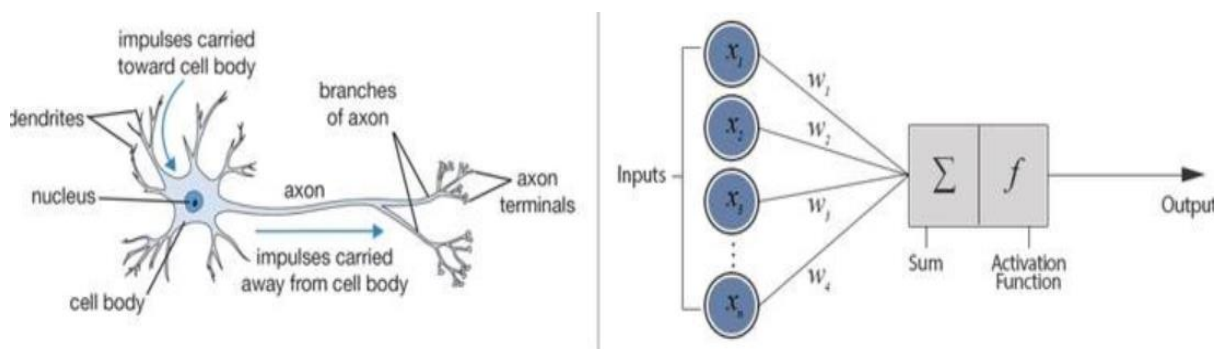
- **Forward Propagation:** Process of passing input data through the network to generate a prediction.
- **Backward Propagation:** Process of calculating gradients to update model parameters during training, using techniques like gradient descent.

2. Understanding Neural Networks

2.1 Structure of a Neural Network

2.1.1 Neurons and connections

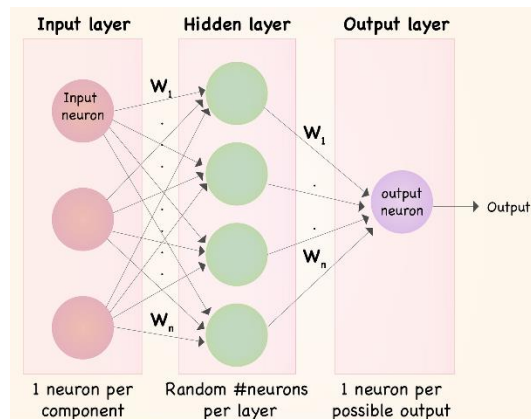
Neural networks consist of interconnected nodes called neurons. Neurons in one layer are connected to neurons in the next layer. Each connection has an associated weight that adjusts during training to learn patterns in the data.



2.1.2 Layers and architectures

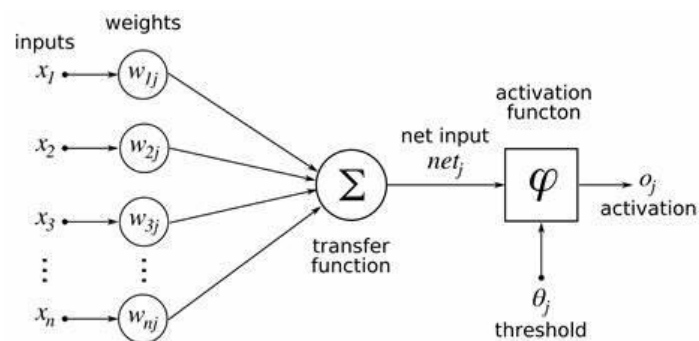
- **Input Layer:** Receives the initial data or features.
- **Hidden Layers:** Intermediate layers between the input and output layers where computations occur.
- **Output Layer:** Produces the final output or prediction.

Neural network architectures vary based on the number of layers (shallow vs. deep) and how neurons are connected (feedforward, recurrent, convolutional).



2.2 Activation Functions

Activation functions introduce non-linearity into the output of neurons, allowing neural networks to model complex relationships in data.



2.2.1 Sigmoid function

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

- Outputs values between 0 and 1, suitable for binary classification tasks.
- Can suffer from vanishing gradients problem.

2.2.2 Tanh function

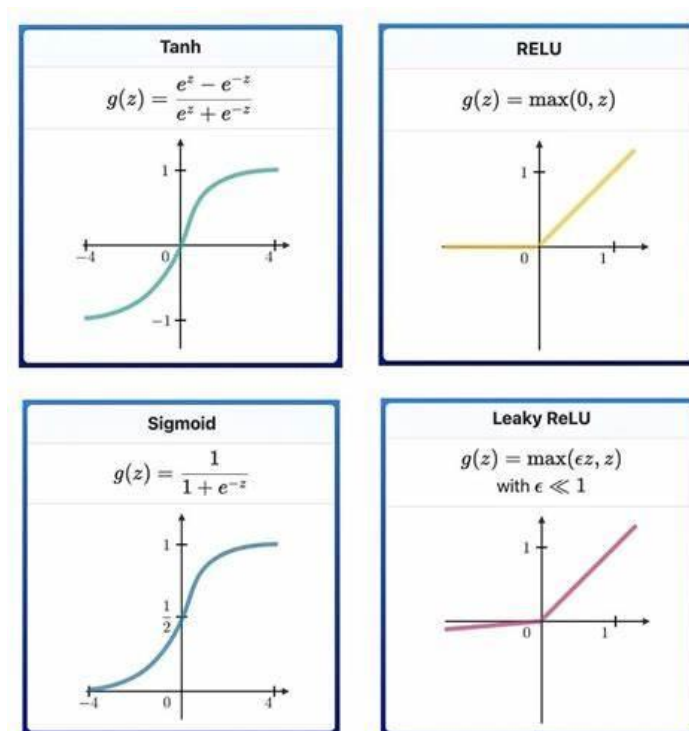
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Outputs values between -1 and 1, centered around zero.
- Helps mitigate the vanishing gradients problem compared to sigmoid.

2.2.3 ReLU (Rectified Linear Unit) and its variants (Leaky ReLU, PReLU)

$$\text{ReLU}(x) = \max(0, x)$$

- Simple and computationally efficient.
- Addresses vanishing gradients by avoiding saturation for positive inputs.
- Variants like Leaky ReLU (introduces a small slope for negative inputs) and Parametric ReLU (learnable slope) improve performance in certain scenarios.



2.3 Loss Functions

Loss functions measure the difference between predicted values and actual values during training, guiding the optimization process.

2.3.1 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Commonly used for regression tasks.
- Sensitive to outliers due to squaring.

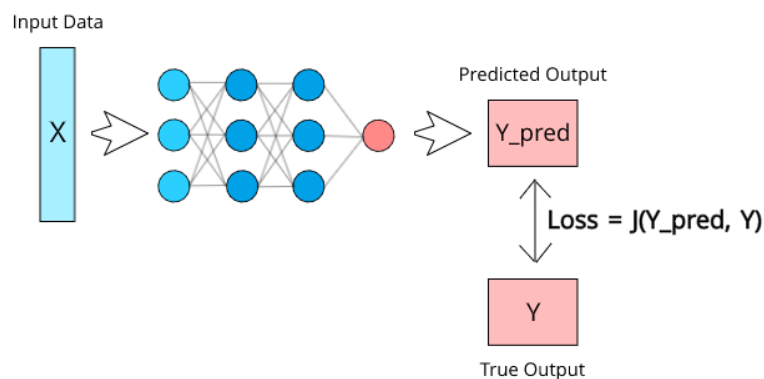
2.3.2 Cross-Entropy Loss (Log Loss)

$$\text{Cross-Entropy Loss} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- Suitable for binary or multiclass classification tasks when outputs are probabilities.
- Penalizes confident wrong predictions heavily.

2.3.3 Choosing the right loss function

- **Regression:** Typically MSE or variants like MAE (Mean Absolute Error) depending on the problem's sensitivity to outliers.
- **Classification:** Cross-Entropy Loss for probability outputs; hinge loss for SVMs if applicable.



3. Introduction to PyTorch for Deep Learning

3.1 Setting up PyTorch for Deep Learning

3.1.1 Installing necessary libraries

First, you need to install PyTorch and other required libraries. Install PyTorch via pip or conda, depending on the environment.

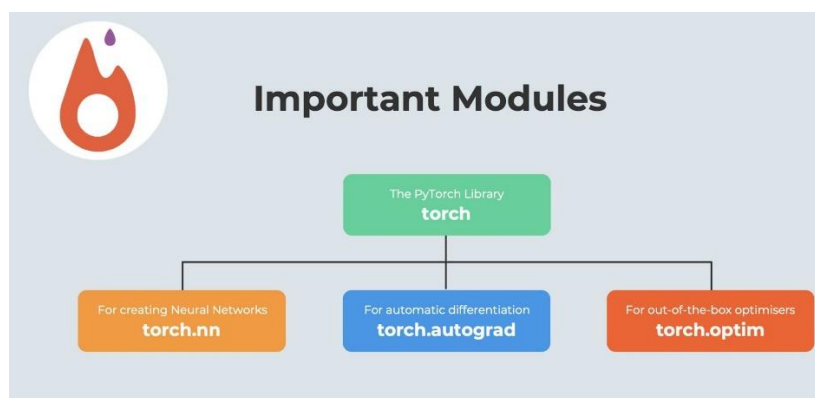
Install PyTorch

pip install torch torchvision torchaudio

3.1.2 Setting up the environment

Once installed, you can start using PyTorch in your Python environment by importing the necessary modules.

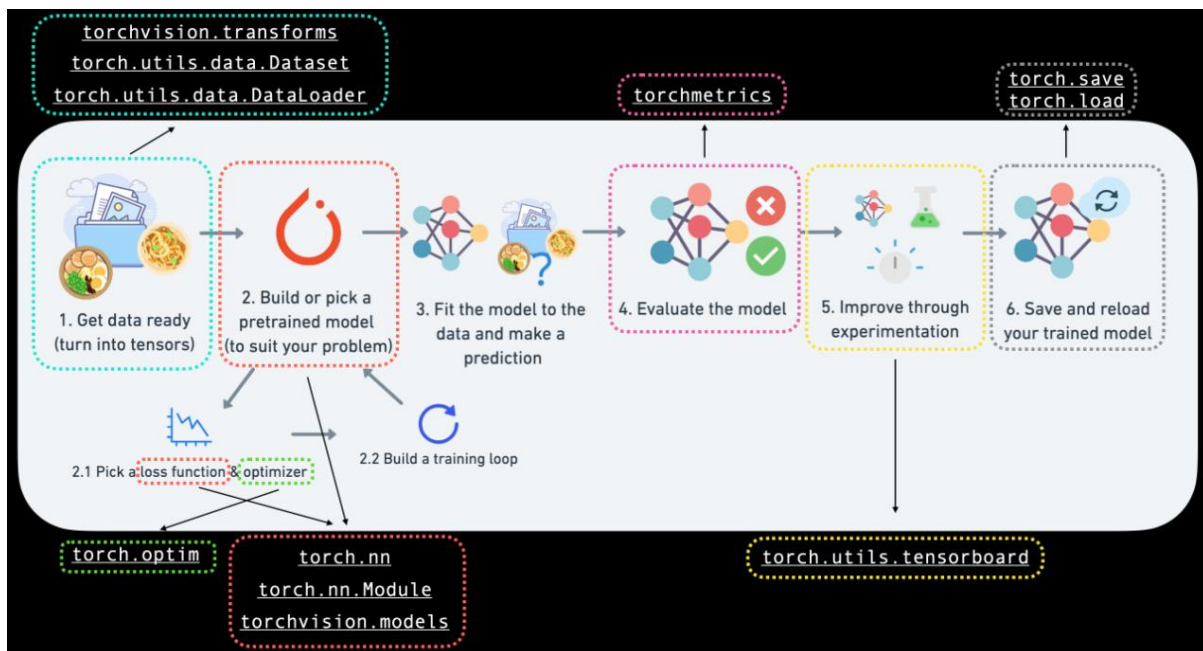
```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
```



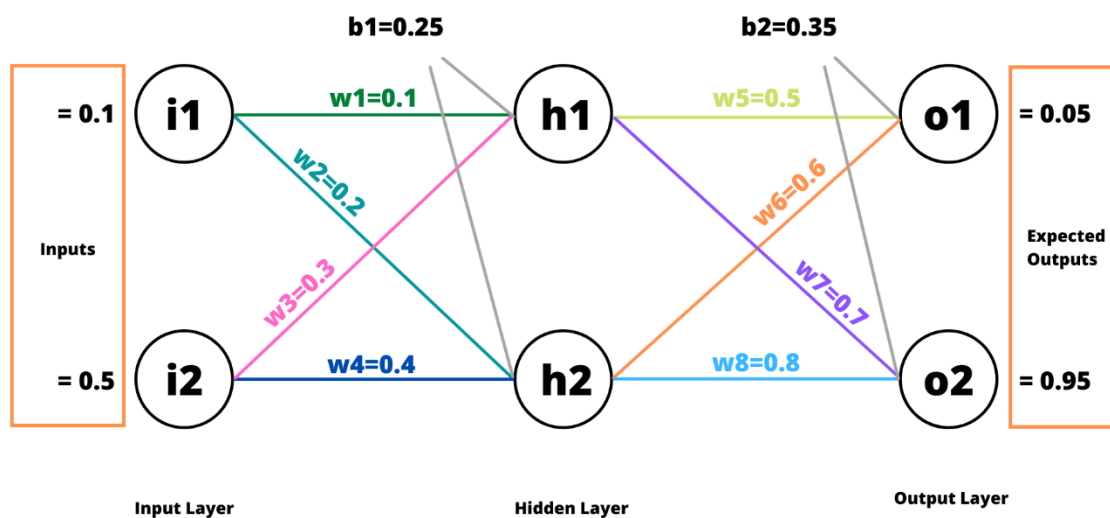
3.2 Basic Workflow in PyTorch

3.2.1 Defining the neural network architecture





Forward & Backward Propagation



There are two units in the Input Layer, two units in the Hidden Layer and two units in the Output Layer.

The $w_1, w_2, w_3, \dots, w_8$ represent the respective weights. b_1 and b_2 are the biases for Hidden Layer and Output Layer, respectively.

Two inputs i_1 and i_2 , and perform a forward pass to compute total error and then a backward pass to distribute the error inside the network and update weights accordingly.

[Neural Networks: Forward pass and Backpropagation | by Ritwick Roy | Towards Data Science](#)

Implementing Neural Network

Define the neural network architecture by creating a class that inherits from `torch.nn.Module` and implements the `__init__` and forward methods.

```
class NeuralNetwork(nn.Module):
    def __init__(self):
        super(NeuralNetwork, self).__init__()
        self.fc1 = nn.Linear(in_features=input_size, out_features=hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(in_features=hidden_size, out_features=output_size)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

# Example usage
input_size = 784 # Example for MNIST data (28x28 images)
hidden_size = 128
output_size = 10 # Number of classes in MNIST
model = NeuralNetwork()
```

3.2.2 Setting up the loss function and optimizer

You define a loss function to compute the difference between predicted and target values, and an optimizer to update the model parameters based on the computed gradients.

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
```

3.2.3 Training the model

Training involves iterating over the dataset, computing predictions, calculating loss, backpropagating gradients, and updating weights.

```
num_epochs = 10
for epoch in range(num_epochs):
    for batch_idx, (data, targets) in enumerate(train_loader):
        # Forward pass
        outputs = model(data)
        loss = criterion(outputs, targets)

        # Backward pass and optimization
        optimizer.zero_grad()
        loss.backward()
```

```
optimizer.step()

# Print statistics
if (batch_idx+1) % 100 == 0:
    print(f'Epoch [{epoch+1}/{num_epochs}], Step [{batch_idx+1}/{len(train_loader)}],
Loss: {loss.item():.4f}')
```

3.2.4 Evaluating the model

Evaluation involves running the model on unseen data (validation set or test set) to measure its performance.

```
def evaluate_model(model, dataloader):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for data, targets in dataloader:
            outputs = model(data)
            _, predicted = torch.max(outputs.data, 1)
            total += targets.size(0)
            correct += (predicted == targets).sum().item()

    accuracy = correct / total
    print(f'Accuracy on the evaluation set: {100 * accuracy:.2f} %')
```

Example usage

```
evaluate_model(model, test_loader)
```

4. Building and Training Neural Networks with PyTorch

4.1 Defining a Simple Neural Network

4.1.1 Using torch.nn.Module to create a network

Define a neural network by creating a class that inherits from torch.nn.Module.

```
import torch
import torch.nn as nn

class SimpleNN(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, num_classes)
```

```

def forward(self, x):
    x = self.fc1(x)
    x = self.relu(x)
    x = self.fc2(x)
    return x

# Example usage:
input_size = 784 # Example for MNIST data (28x28 images)
hidden_size = 128
num_classes = 10 # Number of classes in MNIST
model = SimpleNN(input_size, hidden_size, num_classes)

```

4.1.2 Adding layers to the network

Layers are added within the `__init__` method of your `nn.Module` subclass.

4.2 Forward and Backward Propagation

4.2.1 Implementing the forward pass

In the forward method, define how data flows through the network layers.

```

# Example forward pass:
x = torch.randn(64, 784) # Example input tensor for 64 samples
outputs = model(x)

```

4.2.2 Computing the loss

Choose an appropriate loss function based on your task.

```
criterion = nn.CrossEntropyLoss()
```

4.2.3 Implementing the backward pass (backpropagation)

In the training loop, compute gradients and update weights.

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

```

# Example backward pass:
optimizer.zero_grad()
loss = criterion(outputs, targets)
loss.backward()
optimizer.step()

```

4.3 Training the Neural Network

4.3.1 Preparing the dataset

Load and preprocess your dataset using torchvision or custom data loaders.

```
import torchvision
import torchvision.transforms as transforms

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

train_dataset = torchvision.datasets.MNIST(root='./data', train=True, transform=transform,
download=True)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, transform=transform,
download=True)

train_loader = torch.utils.data.DataLoader ( dataset = train_dataset, batch_size = 64, shuffle =
True)
test_loader = torch.utils.data.DataLoader(dataset=test_dataset, batch_size=64, shuffle=False)
```

4.3.2 Training loop (epoch, batch, forward pass, backward pass, update weights)

Iterate over epochs and batches to train the model.

```
num_epochs = 10

for epoch in range(num_epochs):
    model.train()
    for batch_idx, (images, labels) in enumerate(train_loader):
        # Forward pass
        outputs = model(images.view(-1, 28*28))
        loss = criterion(outputs, labels)

        # Backward pass and optimization
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # Print statistics
    if (batch_idx+1) % 100 == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Step [{batch_idx+1}/{len(train_loader)}],
Loss: {loss.item():.4f}')
```

4.3.3 Monitoring training progress (loss and accuracy)

Evaluate model performance during training on a validation set.

4.4 Evaluating the Neural Network

4.4.1 Evaluating on the validation set

```
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        outputs = model(images.view(-1, 28*28))
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = correct / total
print(f'Accuracy on the test set: {100 * accuracy:.2f}%')
```

4.4.2 Adjusting hyperparameters

Experiment with learning rate, batch size, number of epochs, etc., to optimize performance.

4.4.3 Avoiding overfitting (early stopping, dropout)

Implement techniques like early stopping or add dropout layers to prevent overfitting.

```
# Example of adding dropout
class SimpleNNWithDropout(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(SimpleNNWithDropout, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5) # Dropout with 50% probability
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

5. Practical Example: MNIST Digit Classification

Introduction to the MNIST Dataset

Dataset Overview and Format

The MNIST dataset is a large database of handwritten digits commonly used for training various image processing systems. It includes:

- 60,000 training images
- 10,000 test images
- Each image is a 28x28 pixel grayscale image of a digit (0-9).

MNIST Code Explanation

Explanation of Epochs and Steps

1. Epochs:

- An epoch is one complete pass through the entire training dataset.
- In this case, the training runs for 3 epochs (`num_epochs = 3`).

2. Steps (Batches):

- The training data is divided into smaller subsets called batches.
- Each step (or iteration) processes one batch of data.
- The number of steps per epoch is determined by the total number of training samples divided by the batch size.
- Given `batch_size = 256` and using the MNIST dataset which typically has 60,000 training samples, the number of steps per epoch would be approximately $\lceil 60000/256 \rceil = 235$.

3. Loss:

- The loss is a measure of how well the model's predictions match the true labels.
- The lower the loss, the better the model is performing.

- **Dataset Overview and Loading:** The MNIST dataset is loaded and transformed using `torchvision`.
- **Network Architecture:** A simple neural network with two hidden layers is defined.
- **Training:** The model is trained using the SGD optimizer and cross-entropy loss.
- **Evaluation:** The model's accuracy on the test set is evaluated.
- **Visualization:** Test results and performance metrics including a confusion matrix, precision, recall, and F1-score are visualized and printed.

IRIS Code Explanation

1. Inheritance:

- `super(IrisNN, self).__init__()` calls the constructor of the parent class (`nn.Module`), ensuring that the neural network inherits all necessary properties and methods from `nn.Module`.

2. Layer Definitions:

- `self.fc1 = nn.Linear(input_size, hidden_size)`: This defines the first fully connected layer. It takes inputs of size `input_size` and outputs `hidden_size` features.
 - **input_size**: The number of input features (4 in the case of the Iris dataset, which has four features: sepal length, sepal width, petal length, and petal width).
 - **hidden_size**: The number of neurons in the hidden layer. This is a hyperparameter you can choose (10 in this example).
- `self.relu = nn.ReLU()`: This defines the ReLU (Rectified Linear Unit) activation function, which introduces non-linearity to the model.
- `self.fc2 = nn.Linear(hidden_size, num_classes)`: This defines the second fully connected layer. It takes inputs of size `hidden_size` and outputs `num_classes` features.
 - **hidden_size**: The number of neurons in the hidden layer.
 - **num_classes**: The number of output classes (3 in the case of the Iris dataset, which has three classes: setosa, versicolor, and virginica).

3. Process

- The network has two fully connected layers (`fc1` and `fc2`) and one ReLU activation function between them.
- The `input_size` is the number of features in the input data.
- The `hidden_size` is the number of neurons in the hidden layer.
- The `num_classes` is the number of output classes.
- The forward method defines how data passes through the network: `input -> fc1 -> ReLU -> fc2 -> output`.